

Chapter 1

OOP Part C — Virtual, Override, Abstract, Sealed

ENEL712 Embedded Systems Design — Lecture 17

Summary

Polymorphism mechanics in C#: virtual methods and override, the protected modifier, abstract methods/classes, and sealed restrictions.

Topics Covered

- Virtual methods and the override keyword
- Polymorphism (many forms)
- Protected access modifier
- Abstract methods and classes
- Sealed classes and methods

1.1 Virtual Methods and the override Keyword

- Virtual methods let a base class supply a default implementation that derived classes may replace.
- Classic example: ToString() inherited from System.Object — base returns the class name; derived classes override to return state.
- Syntax: base method uses virtual; derived method uses override; signatures must match exactly.
- Virtual methods cannot be private. Redefining a virtual method without override hides it (compiler warning).

1.2 Polymorphism (Many Forms)

- Single method call can execute different logic depending on the actual runtime type.

- Calls via a base reference dispatch to the most-derived override at runtime.
- Distinct from overloading, which is compile-time signature matching.

1.3 Protected Access Modifier

- Middle ground between public (everyone) and private (same class only).
- Accessible to the defining class and any derived classes.
- Useful for sharing state with derived classes while staying encapsulated from external code.
- Example: Circle has protected double radius; — BottleCap can use radius for area without exposing it publicly.

1.4 Abstract Methods and Classes

- Abstract methods have no implementation in the base class — derived classes must override.
- Mandatory: not optional like virtual methods.
- Only allowed inside abstract classes — abstract classes cannot be directly instantiated.
- Example: Shape declares abstract void area(); — Circle, Square, Triangle each provide their own.

1.5 Sealed Classes and Methods

- Sealed class: no other class can inherit from it — used for security/stability/finalisation.
- Sealed method on an override: no further derived class can override that specific method.
- Useful when a design must remain constant or stable.

1.6 Use Cases & Applications

- Logging: override ToString() on every domain class so log messages and the debugger show meaningful state automatically.
- Plug-in shapes/strategies: an abstract Shape with abstract Area() lets you add new shapes without changing the renderer.
- Library APIs: marking a class sealed prevents users from subclassing in ways the author hasn't anticipated; useful for value-like types.
- Protected helpers: a base ConfigStore can expose a protected ReadRaw() to subclasses so each format (JSON, INI, ...) shares the file-handling code without exposing it publicly.

1.7 Worked Example: Shape Hierarchy with Polymorphism

```
public abstract class Shape
{
    public abstract double Area();
    public override string ToString() =>
        $"{GetType().Name}: area = {Area():F2}";
}
```

```

public class Circle : Shape
{
    public double R { get; }
    public Circle(double r) { R = r; }
    public override double Area() => Math.PI * R * R;
}

public sealed class Square : Shape
{
    public double Side { get; }
    public Square(double s) { Side = s; }
    public override double Area() => Side * Side;
}

Shape[] shapes = { new Circle(2), new Square(3) };
foreach (var s in shapes) Console.WriteLine(s);    // dynamic dispatch via Area()

```

1.8 Common Pitfalls

- Hiding instead of overriding: if you redeclare a base method without override, the base version still runs through a base reference — the compiler warns about this.
- Calling virtual methods from a base constructor runs the derived override before the derived class is fully initialised — surprising behaviour.
- Sealing a class everyone needs to extend; sealing is a design statement, not a default.
- abstract classes cannot be instantiated directly: `new Shape()` doesn't compile.

Key Definitions

Polymorphism

Single method name dispatching to different logic at runtime depending on object type.

virtual

Modifier marking a base method as overridable by derived classes.

override

Modifier replacing a virtual/abstract method in a derived class.

protected

Access modifier limiting members to the class and its derived classes.

abstract

Modifier marking an incomplete method/class; abstract methods must be overridden.

sealed

Modifier preventing inheritance (on a class) or further override (on a method).

Static (Compile-time) Polymorphism

Overload resolution decided at compile time based on argument types.

Dynamic (Runtime) Polymorphism

Method dispatch decided at runtime using the actual object's type, enabled by virtual/override.

Liskov Substitution Principle

Derived objects should be usable wherever a base reference is expected, without surprising the caller.

Interface

C# construct similar to a fully abstract class; supports multiple inheritance of contracts.

Sources

- [Lecture 17.pdf](#)